

UNICA
UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAIfer Lab
Joint lab on Safety and Security of AI

Performance evaluation

Giorgio Fumera, Ambra Demontis, Maura Pintor
Pattern Recognition and Applications Lab
Department of Electrical and Electronic Engineering
University of Cagliari (Italy)

Artificial Intelligence, academic year 2025/2026

Performance evaluation

Machine Learning

Evaluating generalization capability

A classifier should have a good **generalization capability**: it should predict the correct label of “most” unseen instances, having seen only a limited set of examples during design.

1. Which **performance measure** defines how effective a classifier is?
2. How to **estimate the generalization capability** before deployment?

Performance as an expected loss

At design time, the instances processed after deployment are unknown:

- they are viewed as joint random variables (X, Y) (X: attribute vector, Y: class label) with an unknown probability density $P(X, Y)$
- the label predicted by the trained classifier $h(\cdot)$ on a random instance is also a random variable, $\hat{Y} = h(X)$

The performance measure is the **expected value** of a **loss function** $\ell(\hat{Y}, Y)$ of the predicted and true labels, with respect to $P(X, Y)$.

The simplest loss is the **0–1 loss**: $\ell(\hat{Y}, Y) = 0$ if $\hat{Y} = Y$, 1 otherwise. Its expected value is the misclassification probability $P(h(X) \neq Y)$.

Estimating the expected loss

Since $P(X, Y)$ is unknown, any measure can only be **estimated**, with the frequentist definition of probability, on n labeled examples $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$:

$$E[\ell(\hat{Y}, Y)] \approx \frac{1}{n} \sum_{i=1}^{n} \ell(h(\mathbf{x}_i), y_i)$$

Measures of classification performance

Here are some performance metrics (↓: the lower, the better; ↑: the higher, the better):

- **error rate**: fraction of misclassified instances in a given set (↓)
- **accuracy**: fraction of correctly classified instances(↑)

$$\texttt{error_rate} = 1 - \texttt{accuracy}$$

The confusion matrix of a classifier

The **confusion matrix** C of an m-class problem is an $m \times m$ matrix whose element c_{ij} is the number of instances of **true** class j (in a given set) **predicted** as class i.

Example: handwritten digit classifier ($m = 10$ classes, labeled $0, 1, \ldots, 9$), on a set of 1000 instances (100 per class):

predicted class \ true class	0	1	2	3	4	5	6	7	8	9
0	**93**	1	0	0	0	1	0	0	1	0
1	0	**95**	4	1	0	1	1	0	0	1
2	2	0	**91**	0	0	0	0	1	0	0
3	0	1	0	**98**	0	0	1	0	0	1
4	1	0	1	0	**100**	0	0	0	0	0
5	0	0	0	0	0	**94**	0	0	1	0
6	2	1	3	0	0	1	**97**	1	0	2
7	0	0	0	0	0	0	0	**96**	0	0
8	1	2	1	0	0	2	1	1	**97**	4
9	1	0	0	1	0	1	0	1	1	**92**

- correct classifications are on the diagonal: `accuracy` $= 953/1000 = 95.3\%$
- errors are off the diagonal: `error_rate` $= 47/1000 = 4.7\%$

Performance measures for two-class problems

Many two-class problems are **detection** tasks in the input data:

- spam filtering (detecting spam e-mails)
- medical diagnosis (e.g., detecting a given conditino from X-ray, body scan or MRI)
- malware detection (e.g., in PDF documents or mobile applications)

By convention, the class of the “objects” to detect (e.g., spam e-mails) is the **positive** class; the other one (e.g., legitimate e-mails) is the **negative** class.

Is accuracy enough?

Dataset of 1000 e-mails, 10 are spam (`positive`, 1%), 990 are legitimate (`negative`).

A classifier that always predicts `negative`

		true class: `positive`	true class: `negative`
predicted	`positive`	**TP** $= 0$	**FP** $= 0$
	`negative`	**FN** $= 10$	**TN** $= 990$

$$\texttt{accuracy} = \frac{0 + 990}{1000} = 99\%$$

yet it **detects no spam at all**

A classifier that does detect spam

		true class: `positive`	true class: `negative`
predicted	`positive`	**TP** $= 6$	**FP** $= 20$
	`negative`	**FN** $= 4$	**TN** $= 970$

$$\texttt{accuracy} = \frac{6 + 970}{1000} = 97.6\%$$

detects 60% of the spam, **lower accuracy**

With very unbalanced classes the accuracy is dominated by the majority class, and does not characterize how good the classifier is at **detecting**.

Performance measures for two-class problems

The confusion matrix for detection problems has this form:

- **TP** (true positives): positive instances correctly labeled as positive
- **FP** (false positives): negative instances misclassified as positive
- **TN** (true negatives): negative instances correctly labeled as negative
- **FN** (false negatives): positive instances misclassified as negative

		true class: positive	true class: negative
predicted	positive	TP	FP
predicted	negative	FN	TN

Performance measures for two-class problems

The error rate is:

$$\texttt{error_rate} = \frac{\mathbf{FP} + \mathbf{FN}}{\mathbf{TP} + \mathbf{TN} + \mathbf{FP} + \mathbf{FN}}$$

FP and **FN** errors are weighted equally, but their consequences may differ.

- a spam e-mail labeled as legitimate (**FN**) is annoying, since the user has to remove it manually; a legitimate e-mail labeled as spam (**FP**) may cause important communications to be missed
- a healthy patient diagnosed as ill (**FP**) causes trouble, but can be repaired with further tests; a sick patient diagnosed as healthy (**FN**) may not receive the necessary treatment promptly

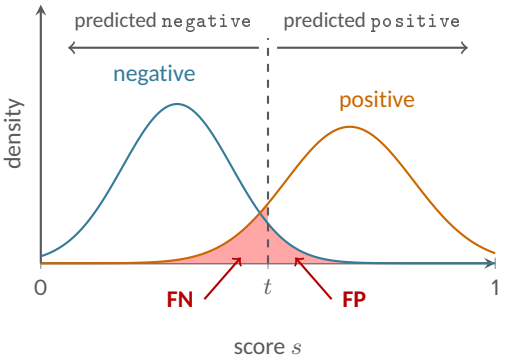
Performance measures for two-class problems

Some classifiers (e.g., decision trees) output a class label. Others (e.g., neural networks) output a **score**, e.g., a real number $s \in [0, 1]$.

A **decision threshold** t turns the score into a predicted label y:

$$s \geq t \Rightarrow y = \texttt{positive}$$
$$s < t \Rightarrow y = \texttt{negative}$$

Increasing t **decreases** the **FP** and **increases** the **FN**. The threshold trades off the two errors, depending on the application.

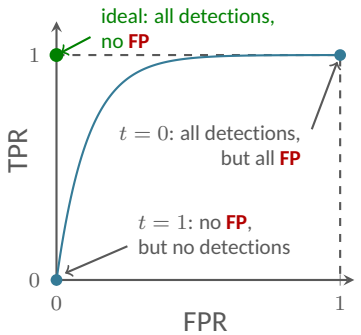


Performance measures for two-class problems

The **Receiver Operating Characteristic (ROC) curve**[1] depicts the true positive rate (TPR) vs the false positive rate (FPR), e.g., as a function of the decision threshold:

$$\text{TPR} = \frac{\textbf{TP}}{\textbf{TP} + \textbf{FN}} \qquad \text{FPR} = \frac{\textbf{FP}}{\textbf{TN} + \textbf{FP}}$$

- TPR: fraction of positive instances correctly labeled as positive (a.k.a. **sensitivity** or **recall**)
- FPR: fraction of negative instances wrongly labeled as positive ($1 -$ FPR is the **specificity**)

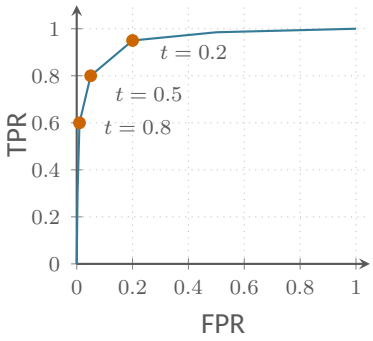


[1]The ROC curve was first developed by electrical engineers and radar engineers during World War II for detecting enemy objects in battlefields.

The ROC curve: an example with spam e-mails

t	TP, FN	FP, TN	(FPR, TPR)
0.8	60, 40	9, 891	(0.01, 0.60)
0.5	80, 20	45, 855	(0.05, 0.80)
0.2	95, 5	180, 720	(0.20, 0.95)

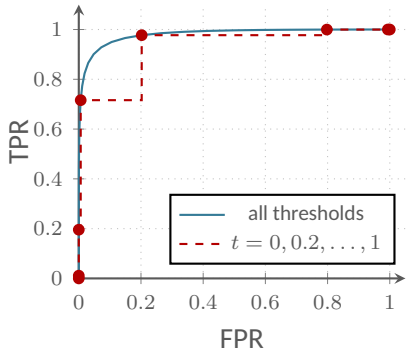
In practice, we usually **fix the FPR** (the false positives we can tolerate) and **measure the TPR at that FPR** (**TPR@FPR**):



e.g., at FPR $= 1\%$ the filter detects TPR $= 60\%$ of the spam.

A common mistake: the staircase ROC curve

A common mistake is to set t to a **fixed set of values** (e.g., $0, 0.2, \ldots, 1$), compute **FP** and **TP** at each of them, and using them as the ROC points.



If you see a **staircase ROC**, come back to this slide and check that you are using the **scores** and not the **predicted labels**!!!

Expected classification cost

When the consequences of errors can be quantified (e.g., in money), each entry c_{ij} of the confusion matrix has a **cost** λ_{ij}:

- correct classifications usually have zero cost, i.e. $\lambda_{ii} = 0$ $(i = 1, \ldots, m)$
- misclassifications (λ_{ij} $(i \neq j)$) may have specific costs depending on the consequences of error

The performance measure is then the **expected classification cost**, estimated from the confusion matrix on a set of n labeled instances:

$$\frac{\sum_{i=1}^{m} \sum_{j=1}^{m} \lambda_{ij}\, c_{ij}}{n}$$

This is the estimate of the expected value of the **cost loss** $\ell(\hat{Y}, Y) = \lambda_{\hat{Y}Y}$: each of the c_{ij} instances of the entry (i, j) contributes λ_{ij}.

Expected classification cost

If all misclassifications have the same cost, $\lambda_{ij} = \lambda$ for $i \neq j$, and correct classifications cost nothing ($\lambda_{ii} = 0$), the expected cost is **proportional to the error rate**:

$$\frac{\sum_{i=1}^{m} \sum_{j=1}^{m} \lambda_{ij}\, c_{ij}}{n} = \lambda \frac{\sum_{i=1}^{m} \sum_{j=1,\, j \neq i}^{m} c_{ij}}{n} = \lambda \times \texttt{error_rate}$$

Expected classification cost: an example

Costs: a legitimate e-mail labeled as spam (**FP**) may cause important communications to be missed, $\lambda_{\text{FP}} = 10$; a spam e-mail left in the inbox (**FN**) is only annoying, $\lambda_{\text{FN}} = 1$; correct decisions cost 0.

Cost matrix λ_{ij}

predicted \ true class	positive	negative
positive	$\lambda_{\text{TP}} = 0$	$\lambda_{\text{FP}} = 10$
negative	$\lambda_{\text{FN}} = 1$	$\lambda_{\text{TN}} = 0$

$$\text{cost} = \frac{10 \cdot \textbf{FP} + 1 \cdot \textbf{FN}}{n}$$

t	FP	FN	error_rate	cost
0.9	2	70	7.2%	**0.090**
0.8	9	40	**4.9%**	0.130
0.5	45	20	6.5%	0.470
0.2	180	5	18.5%	1.805

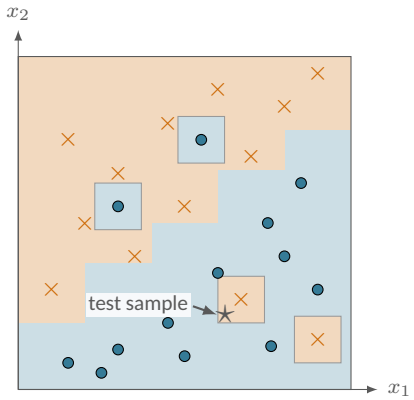
e.g., $t = 0.9$: $\dfrac{10 \cdot 2 + 1 \cdot 70}{1000} = 0.09$

The **lowest error rate** is at $t = 0.8$, but the **lowest cost** is at $t = 0.9$ (due to higher λ_{FP}).

Estimating the generalization capability

Any performance measure can be estimated on a set of labeled examples. Using the

same set that trained the classifier will **underestimate** the true error, because learning algorithms minimize the error on the training examples (over-fitting).



The hold-out technique

Randomly split the labeled examples collected for design into two subsets:

- a **training set** (e.g., 70% of the examples), used by the learning algorithm
- a **testing set** (the rest), used to estimate the generalization capability of the trained classifier on instances unseen by the learning algorithm

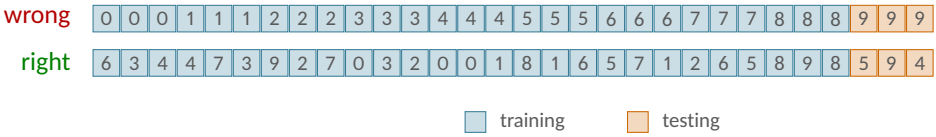
Averaging over several random splits makes the estimate more reliable.

Ensure that the **classes are randomly distributed** in the two sets. If the examples are sorted by class and you take the first ones for training, you could train on classes 0 to 8 and test only on class 9, which the classifier has never seen.



Hold-out: a common mistake

How the examples are split matters. Example with 10 classes, 3 examples per class, 90% for training and 10% for testing:



Ensure that the **classes are randomly distributed** in the two sets. If the examples are sorted by class and you take the first ones for training, you could train on classes 0 to 8 and test only on class 9, which the classifier has never seen.

Model selection: the validation set

Often several classifiers must be compared (e.g., two DTs).

We need to estimate the generalization capability (avoiding training set) and at the same time evaluate the final performance on an unseen set.

Split the labeled examples into **three** sets:



Example, two decision trees:

	validation error	testing error
tree A	12%	
tree B	**9%**	10%

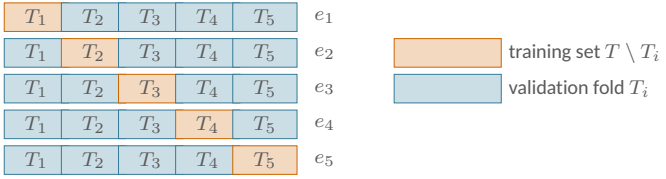
Tree B is chosen on the **validation** set; the **testing** error of the chosen DT (10%) estimates the generalization.

Never choose between two classifiers using the **testing set**! It is no longer "unseen". Use the testing set only once, at the end.

The cross-validation technique

k-fold cross-validation provides a method to use all samples available:

1. randomly split the available examples T into k disjoint, equally sized subsets $T_1, \ldots, T_k$, called **folds** (typically $k = 5$ or 10)
2. for $i = 1, \ldots, k$: train a classifier on $T \setminus T_i$, and estimate its performance e_i on T_i
3. estimate the generalization capability as the average over the folds, $\frac{1}{k} \sum_{i=1}^{k} e_i$



The deployed classifier is trained on the whole T.
It provides a more accurate estimate (less biased) but also requires k trainings.